

## Introduction

---

Palindrome problems show up constantly in coding interviews because they test something more specific than "do you know a palindrome is." They test whether you can see structure in a string that most people only see as a flat sequence of characters. Smallest Palindromic Rearrangement I is a perfect example. On the surface it looks like a sorting or brute-force problem. Underneath, it's really a lesson in symmetry — and once you see it, you'll start noticing the same "only think about half the problem" pattern in run-length encoding, DNA sequence correction, and even autocomplete systems.

This problem is also a great one to learn early because the trap is so tempting. Your first instinct will almost certainly be to generate every possible arrangement of the letters and filter for the palindromes. That instinct isn't wrong — it's just going to cost you the interview the moment the interviewer says "what if the string has 100,000 characters?" Learning to recognize when a full-arrangement approach quietly becomes impossible is one of the most transferable skills in this entire subject.

## Problem Overview

---

Here's the problem in plain terms: you're given a string that is already a palindrome — meaning it reads the same forwards and backwards, like `babab` or `tacocat`. Your job is to rearrange its letters (you can't add, remove, or change any letter) into the alphabetically smallest string that is *also* a palindrome.

"Alphabetically smallest" works the same way it does when you sort words in a dictionary. `abbba` comes before `babab` because `a` comes before `b`. You're not inventing a new string — you're just finding the best possible ordering of the letters you were already given.

## Example

---

Take the string `babab`. It has three `b`s and two `a`s. If we rearrange those same five letters into `abbba`, we still have three `b`s and two `a`s, and the string reads identically forward and backward. Compare `abbba` to other valid palindromic rearrangements like `babab` or `bbabb` — `abbba` is the smallest because its very first character, `a`, beats every other candidate's first character, `b`.

Now try `tacocat`. Counting letters gives us `t:2`, `a:2`, `c:2`, `o:1`. The only letter with an odd count, `o`, must sit in the exact middle of the palindrome — every other letter has to be split evenly

across the two mirrored halves. Arranging the remaining letters alphabetically for the left half gives `act`, so the finished palindrome is `actotca`.

## Intuition

---

Here's the mental shift that makes this problem easy: **a palindrome is completely determined by its left half**. The right half isn't independent information — it's a forced mirror image of the left half. If you imagine a mirror standing in the exact center of the string, whatever you place on the left automatically appears (flipped) on the right.

That means the entire problem — "find the smallest palindrome" — quietly shrinks into a much smaller problem: "find the smallest possible *left half*." Whatever you build there gets copied for free.

So how do you build the smallest possible left half? The same way you'd build the smallest possible *string* out of a bag of letters: use the smallest letters first. If you have two `a`s and three `b`s, you want `a` appearing before `b` in your left half, because `a` is alphabetically smaller.

There's one wrinkle. Since the original string is already a palindrome, at most one letter can have an odd count (that's just a mathematical property of palindromes — every letter except possibly one has to have a partner on the other side of the center). That one odd-count letter can't go in either half — it belongs in the very middle, alone. Everything else gets split evenly: half its count goes into your left-half build, and the mirror handles the rest automatically.

Once you see this, the algorithm writes itself: count the letters, pull out the (at most one) odd-count letter as the middle character, take half of every other count in alphabetical order to build the left half, then glue it all together as `left + middle + reverse(left)`.

## Brute Force Solution

---

**Idea:** Generate every distinct permutation of the string's letters, keep only the ones that are palindromes, sort the survivors, and return the smallest one.

**Algorithm:** 1. Generate all unique permutations of the string. 2. For each permutation, check if it reads the same forwards and backwards. 3. Collect all palindromic permutations. 4. Return the smallest one alphabetically.

**Advantages:** Dead simple to reason about. Works fine for tiny strings, and it's a natural starting point if you're just trying to understand the problem.

**Disadvantages:** The number of permutations of a string grows as  $n!$  ( $n$  factorial). For `tacocat` (7 letters), that's already 5,040 arrangements — annoying but doable. For a 20-letter string, it's over 2 quintillion. For LeetCode's actual constraints (strings up to 100,000 characters), this approach doesn't just get slow — it becomes mathematically impossible to finish, even with unlimited computing power.

**Time complexity:**  $O(n! \times n)$  — factorial arrangements, each requiring  $O(n)$  to check and compare.

**Space complexity:**  $O(n! \times n)$  to store all the permutations.

## Optimal Solution

The Half and Mirror approach turns the factorial blowup into a single linear pass. Here's the breakdown:

| Step                    | What happens                                                              | Why it matters                                              |
|-------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------|
| 1. Count                | Tally every letter's frequency                                            | Tells us exactly how many of each letter we're working with |
| 2. Find the odd one out | At most one letter will have an odd count                                 | That letter is forced into the exact middle                 |
| 3. Build the half       | For each letter (alphabetical order), take <code>count // 2</code> copies | Smallest letters first = smallest possible left half        |
| 4. Mirror               | <code>answer = half + middle + reversed(half)</code>                      | The right side is never computed — it's just a copy         |

Because we iterate through letters in alphabetical order ( `a` to `z` ), the left half we build is automatically the smallest possible arrangement of those characters — there's no separate sorting step needed for the output, since we're constructing it in order from the start.

## Python Code

```
from collections import Counter

def smallest_palindrome(s: str) -> str:
    """Rearrange a palindrome's letters into the smallest palindrome possible."""
    counts = Counter(s)

    half_chars = []
    middle_char = ""

    for letter in sorted(counts):
        count = counts[letter]
        if count % 2 == 1:
            middle_char = letter
        half_chars.append(letter * (count // 2))
```

```
half = "".join(half_chars)
return half + middle_char + half[::-1]
```

## Code Walkthrough

- `Counter(s)` builds a frequency map in one pass — `{'t': 2, 'a': 2, 'c': 2, 'o': 1}` for `tacocat`.
- `sorted(counts)` iterates the *keys* of the counter alphabetically, guaranteeing we place smaller letters earlier in the left half.
- `count % 2 == 1` catches the single odd-count letter (guaranteed to be at most one, since the input is already a palindrome) and saves it as `middle_char`.
- `letter * (count // 2)` appends half of that letter's total count to the half-string — integer division naturally drops the odd leftover, which is already accounted for by `middle_char`.
- `half[::-1]` reverses the completed left half to produce the mirrored right side — no extra logic needed for letters that never got an odd count.
- Final assembly `half + middle_char + half[::-1]` glues the three pieces into the finished palindrome. If there's no middle character, `middle_char` is just an empty string and disappears from the concatenation.

## Dry Run

Input: `tacocat`

1. `Counter("tacocat")` → `{'t': 2, 'a': 2, 'c': 2, 'o': 1}`
2. Iterate sorted keys: `a, c, o, t`
3. `a` : count 2, even → append `"a"` (1 copy)
4. `c` : count 2, even → append `"c"` (1 copy)
5. `o` : count 1, odd → set `middle_char = "o"`, append `""` (0 copies)
6. `t` : count 2, even → append `"t"` (1 copy)
7. `half = "act"`
8. Result: `"act" + "o" + "tca" → "actotca"`

Matches the expected answer from the video.

## Complexity Analysis

**Time:**  $O(n)$  — one pass to build the counter, one pass over the fixed 26-letter alphabet to build the half (constant, since there are only 26 possible letters), and one pass to reverse and concatenate. All of this is linear in the length of the input string.

**Space:**  $O(n)$  — the output string and the half-string are each proportional to the input length; the counter itself only ever holds at most 26 entries.

These bounds are correct because every step touches each character (or each of the 26 possible letters) exactly once — there's no nested loop, no re-scanning, and no exponential branching anywhere in the algorithm.

## Alternative Solutions

---

You could technically sort the entire string, split it into two mirrored halves, and check parity separately — but that's strictly more work than counting, since sorting costs  $O(n \log n)$  versus the  $O(n)$  counting approach. Counting wins because we only care about *frequencies*, not the actual relative order of every character in the original string.

## Edge Cases

---

- **Single character** ( "z" ): counts to `{ 'z': 1 }`. No even-count letters, so `half` stays empty and `middle_char = "z"`. Result: "z".
- **All identical letters** ( "aaaa" ): `half = "aa"`, no middle character. Result: "aaaa".
- **Already the smallest palindrome**: rebuilding it from its own letter counts reproduces the identical string — the algorithm is idempotent.
- **Even-length input with no odd-count letters** ( "abba" ): `middle_char` stays empty, and `half + "" + half[::-1]` still produces a correct even-length palindrome.
- **Long strings with many repeated letters** (100,000 characters): performance stays linear; nothing about the algorithm depends on string length beyond the single counting and building passes.

## Common Mistakes

---

1. **Forgetting that at most one letter can have an odd count** and mishandling multiple "middle" candidates — this can't happen with valid input, but skipping the check entirely leads to bugs if you reuse this logic for non-palindromic inputs.
2. **Sorting the string directly** instead of sorting on the *counted letters* — this wastes time and can scramble the pairing logic.

3. **Appending the full count instead of half the count** to the left-side builder, which doubles the letters in the final output.
4. **Forgetting to reverse the half** before mirroring, producing a string that isn't actually a palindrome.
5. **Placing the middle character in the wrong position** — it must sit between the half and its reverse, not at the start or end.
6. **Assuming brute force is "fine" without checking constraints** — 5,040 permutations feels manageable, but the same code silently becomes unusable at scale.

## Interview Questions

---

- "What if the input string isn't guaranteed to be a palindrome — can you still rearrange it into one?" (Hint: check if more than one letter has an odd count; if so, it's impossible.)
- "Can you do this in-place, without building a new string?"
- "How would this change if you needed the *largest* palindrome instead of the smallest?"
- "What's the time complexity if the alphabet isn't fixed at 26 letters (e.g., Unicode)?"

## Similar Problems

---

- **LeetCode 409 — Longest Palindrome**: uses the same "at most one odd count" insight to determine the longest palindrome constructible from a set of letters.
- **LeetCode 1400 — Construct K Palindrome Strings**: extends the odd-count counting logic to multiple palindromes at once.
- **LeetCode 267 — Palindrome Permutation II**: generates all distinct palindromic permutations, a good next step after mastering the single-answer version.
- **Smallest Palindromic Rearrangement II** (tomorrow's video in this series): builds directly on today's mirroring technique for a harder variant.

## Key Takeaways

---

A palindrome is fully described by its left half — the right half is free once you have the left. Counting letters and splitting each count in half turns an exponential brute-force search into a single linear pass. Whenever you're stuck on a symmetry-based problem, ask yourself: "am I solving twice the work I actually need to?"

## Watch the Video

---

Prefer to see this walked through visually, with the dry run drawn out step by step? Watch the full explanation here: {LINK}

## About the Series

---

This walkthrough is part of **Fun with Learning Technology**, a daily series that breaks down LeetCode problems into their core intuition first — no memorized templates, no rushing to code. Each video builds on ideas from the ones before it, so patterns like Half and Mirror keep showing up in smarter, harder forms as the series continues.

## Call To Action

---

If this clicked for you, the video walks through the same dry run with visuals that make the mirroring click even faster — give it a watch and a like on YouTube. Subscribe so you don't miss tomorrow's follow-up, which pushes this same trick into a tougher rearrangement problem. If you'd rather read than watch, subscribe to the Substack for the written breakdown of every problem in the series. And if you got stuck anywhere or have a cleaner approach, drop it in the comments — sharing this with someone prepping for interviews helps more than you'd think.

---

## Quick Review

---

### Trick for smallest palindromic rearrangement of same letters?

Half-and-mirror: build smallest half from sorted char counts, then mirror it; odd-count char sits in the center.

### Time complexity of the half-and-mirror approach?

$O(n + k \log k)$ ,  $n$  = string length,  $k$  = distinct characters — counting plus sorting the alphabet.

### Space complexity of this approach?

$O(n)$  for the output string plus  $O(k)$  for the character frequency map.

### Trickiest edge case in this problem?

More than one character with odd count means no palindrome is possible — must detect and return early.

**Why not brute-force every arrangement?**

Factorial blowup ( $n!$  permutations) vs linear counting — brute force is intractable even for short strings.

**Follow-up: how to find the actual smallest permutation, not just a palindrome?**

Different problem — that's next-permutation/greedy digit-DP, not frequency-count mirroring.